

Connexion avant accès

La plateforme exige une connexion avant d'afficher quoi que ce soit. Deux portes :

- **Google** : si la section `[auth]` est configurée, un bouton « Se connecter avec Google » apparaît.
- **Compte CHRUTH** : toujours disponible. Sans Google configuré, c'est la seule porte — chaque membre crée son compte (adresse + mot de passe) depuis l'écran de connexion, puis se connecte. Le mot de passe n'est jamais stocké en clair : seul son hachage vit dans l'espace chiffré du membre.

Dans les deux cas, l'application ne répond qu'à une question : **cette adresse a-t-elle le droit d'entrer ?**

La réponse vit dans `.streamlit/secrets.toml`, un fichier que seul l'administrateur peut modifier — un droit d'accès modifiable depuis l'application protégée ne protégerait rien. Sans `[acces]` (poste local), tout compte authentifié entre.

1. Créer l'identifiant Google (une seule fois, ~10 minutes)

1. Ouvrir <https://console.cloud.google.com/> et créer un projet (nom libre).
2. Menu **APIs et services** → **Ecran de consentement OAuth**. - Type **Externe**, renseigner le nom de l'application et un email de contact. - Dans **Utilisateurs test**, ajouter les adresses qui se connecteront tant que l'application n'est pas publiée.
3. Menu **Identifiants** → **Créer des identifiants** → **ID client OAuth**. - Type : **Application Web**. - **URI de redirection autorisés** : `http://localhost:8501/oauth2callback` (ajouter aussi l'adresse en ligne le jour d'un déploiement, avec le même suffixe `/oauth2callback`).
4. Copier l'**ID client** et le **Code secret du client**.

2. Remplir le fichier de secrets

```
cp .streamlit/secrets.toml.template streamlit/secrets.toml
python -c "import secrets; print(secrets.token_hex(32))" # pour cookie_secret
```

Puis renseigner dans `.streamlit/secrets.toml` : `cookie_secret`, `client_id`, `client_secret`, et les adresses dans `[acces]`.

```
[acces]
emails = ["collaboratrice@exemple.fr"] # liste VIDE = tout compte Google entre
admins = ["ton.adresse@exemple.fr"]
```

3. Verifier

```
streamlit run CHRUTH_APP.py
```

Attendu : un ecran « Se connecter », puis la plateforme. La barre laterale affiche l'adresse connectee et un bouton **Se deconnecter**.

Gerer les acces au quotidien

Objectif	Geste
Ouvrir l'acces a quelqu'un	Ajouter son adresse dans <code>emails</code> , relancer l'app
Retirer un acces	Retirer son adresse de <code>emails</code> , relancer l'app
Ouvrir a tout compte	Laisser <code>emails = []</code> (tout compte authentifie entre)
Retirer le bouton Google	Commenter toute la section <code>[auth]</code> (la connexion reste requise via les comptes locaux)

Une adresse refusee voit un message qui nomme les administrateurs a contacter : un refus sans recours transforme un reglage a corriger en impasse.

Points de vigilance

- `.streamlit/secrets.toml` est ignore par git. Ne jamais le commiter, ne jamais le joindre a une livraison — il ouvre l'acces a l'application.
- En deploiement Streamlit Community Cloud, ces valeurs se saisissent dans **Settings** → **Secrets**, pas dans un fichier.
- Changer `cookie_secret` deconnecte tout le monde. C'est le geste a faire si on soupconne une fuite.